

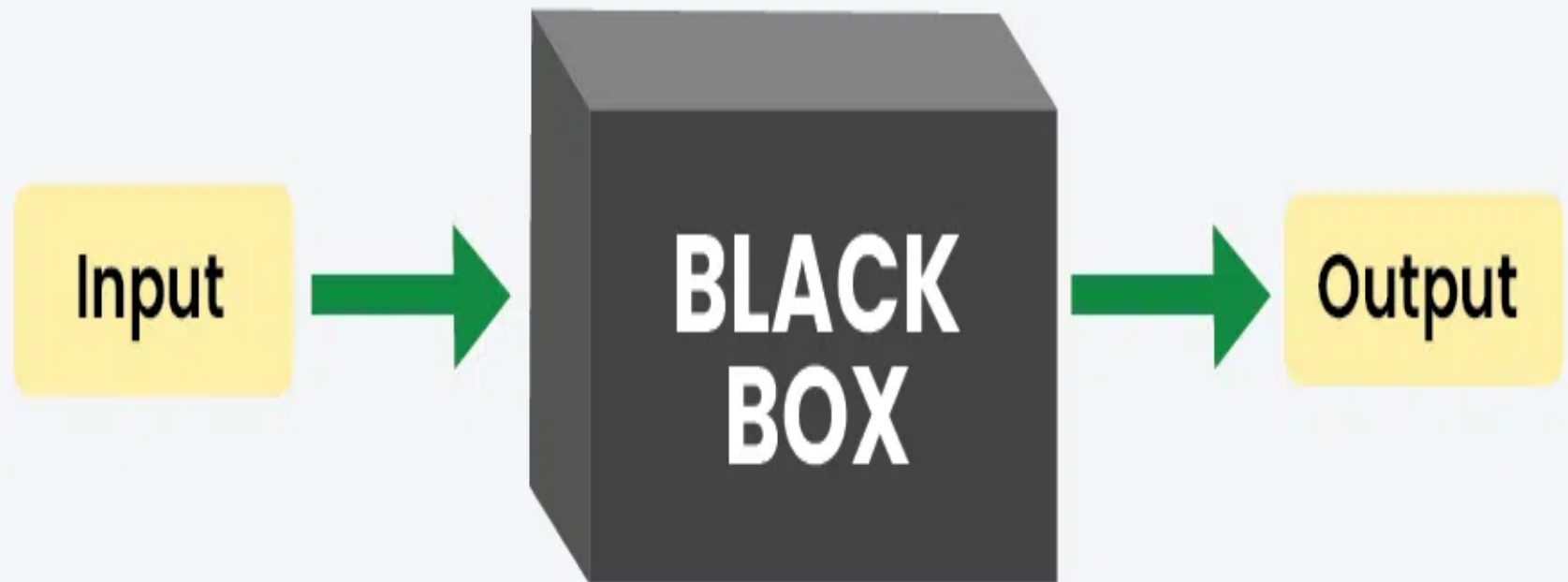
Module - 2

Black box testing techniques

What is Black box testing ?

- Black box testing is a software testing technique where the internal workings or code structure of the system being tested are not known to the tester.
- In other words, the tester focuses solely on the external behaviour of the software, without having access to its internal source code. The name “black box” comes from the idea that the internal workings are hidden or “boxed” from the tester’s view.

Black Box Testing



Key characteristics of black box testing include:

- **Independent Testing:** Black box testing is typically performed by testers who are independent of the development team. This ensures a fresh perspective and helps identify issues that developers might overlook.
- **Requirements-Based Testing:** Testers design test cases based on the software's requirements and specifications, without being concerned about how the code is implemented.
- **Functional Testing:** The main goal of black box testing is to assess the functionality of the software, checking if it meets the expected behaviour and delivers the desired outputs for various inputs.
- **No Knowledge of Internal Code:** Testers do not have access to the source code, architecture, or design details of the software. They interact with the system through its user interfaces or APIs.

Different Types of Black Box Testing :

Black box testing encompasses several types of testing techniques, each with a specific focus and objective. Some of the main types of black box testing include:

- **Functional Testing**: This type of black box testing verifies that the software's functions and features work as expected and adhere to the specified requirements. Testers use functional test cases to validate the application's inputs, outputs, and interactions, without being concerned about the internal code.
- **Non-Functional Testing**: Unlike functional testing, non-functional testing evaluates aspects of the software that are not related to its specific functions. It includes tests for performance, usability, security, scalability, reliability, and other quality attributes.

- **Regression Testing**: Regression testing is performed to ensure that recent changes or updates to the software do not adversely affect existing functionality. Testers use a set of predefined test cases to verify that new features or bug fixes have not introduced new issues.
- **User Interface (UI) Testing**: UI testing focuses on validating the user interface elements of the software, such as buttons, menus, forms, and layout. The goal is to ensure that the UI is user-friendly, consistent, and functions correctly.
- **Usability Testing**: Usability testing assesses the software's user-friendliness and how easily users can interact with it. Testers evaluate factors like navigation, visual appeal, ease of learning, and overall user experience.

Black Box Testing Techniques

- **Boundary Value Analysis**
- **Equivalence Partitioning**
- **State table based testing**
- **Decision Table based Testing**
- **Cause-Effect graphing based Technique**
- **Orthogonal array testing**
- **Error Guessing testing**

1. Boundary Value Analysis (BVA):

Tests the bounds of input ranges, as errors frequently arise on the edge of input limits.

Example: For an input field that accepts values from 1 to 100, boundary values would include:

- Lower boundary: 1
- Just below lower boundary: 0
- Just above upper boundary: 101
- Upper boundary: 100

How Boundary Value Analysis Works ?

- For a given valid input range (minimum value to maximum value), BVA typically involves testing values:
- **At the minimum and maximum boundaries** (valid).
- **Just below the minimum and just above the maximum boundaries** (invalid).
- **Just above the minimum and just below the maximum boundaries** (valid).

This approach efficiently reduces the number of test cases while maximizing test coverage for potential defects.

Example: Online Movie Ticket Purchase

- Consider a website that allows users to purchase a quantity of movie tickets. The requirement specifies a user must buy between **1 and 10 tickets**, inclusive.
- **Valid Range:** 1 to 10
- **Invalid Ranges:** Less than 1, Greater than 10
- Based on the BVA technique, the following test cases should be created, By executing these test cases, a tester can verify that the system correctly handles inputs at its operational limits and appropriately rejects invalid inputs, thereby ensuring the functionality is robust and reliable.

Test Case 	Input Value	Expected Result	Description
TC1	0	Invalid (Error message)	Just below minimum boundary
TC2	1	Valid (Accepts input)	At minimum boundary
TC3	2	Valid (Accepts input)	Just above minimum boundary
TC4	9	Valid (Accepts input)	Just below maximum boundary
TC5	10	Valid (Accepts input)	At maximum boundary
TC6	11	Invalid (Error message)	Just above maximum boundary

- **Single Fault Assumption :**

Single Fault Assumption (SFA) is the assumption that **only one fault exists in a system or circuit at any given time**, while all other components operate correctly.

Example: Traffic Signal System

Imagine a **traffic signal controller** with three lights:

Red ,Yellow, Green

Each light is controlled by a digital circuit.

Applying Single Fault Assumption :

- During testing, engineers assume **only one fault occurs at a time**.
- **Case : Green light stuck OFF**
- Fault assumed: **Green bulb/circuit is faulty**
- Red and Yellow are assumed to work correctly
- Observation:
 - Green never turns ON → fault detected
- This follows SFA because **only one fault** is considered.

For n variable to be checked:

- Assume a system has **n input variables**, and each variable has:
- a **minimum value**
- a **maximum value**

For **Boundary Value Coverage**, we test:

For each input variable:

- **Min boundary**
- **Min + 1**
- **Max – 1**
- **Max boundary**

That gives **4 boundary values per input variable**.

- One **nominal (normal) test case** is added
- In this test case, **all input variables are set to typical (middle) values**
- This ensures the system works correctly under normal conditions

Total Test Cases

- Total test cases= **$4n+1$**

Single Fault Assumption (SFA) is applied in BVA because:

- **Most defects occur at boundaries**
 - Experience shows that errors usually happen near **minimum, maximum, and edge values**
 - BVA assumes **one boundary-related fault at a time**
- **Simplifies test case design**
 - Testing one fault at a time makes it easier to:
 - Identify the cause of failure
 - Map a failure to a specific boundary
- **Reduces test cases**
 - Without SFA, combinations of boundary faults would be tested
 - SFA avoids **combinatorial explosion**
- **Efficient fault isolation**
 - When a test fails, SFA helps testers quickly pinpoint **which boundary condition caused the fault**
- **Practical and cost-effective**
 - In real projects, multiple simultaneous boundary faults are rare
 - SFA balances **test coverage and effort**

Problem: Consider a Program for determining the Previous Date.

Input: Day, Month, Year with valid ranges as-

$1 \leq \text{Month} \leq 12$

$1 \leq \text{Day} \leq 31$

$1900 \leq \text{Year} \leq 2000$

Design Boundary Value Test Cases.

- **Solution:**
- i) Taking the year as a Single Fault Assumption i.e. year will be having values varying from 1900 to 2000 and others will have nominal values.

Test Cases	Month	Day	Year	Output
1	6	15	1900	14 June 1900
2	6	15	1901	14 June 1901
3	6	15	1960	14 June 1960
4	6	15	1999	14 June 1999
5	6	15	2000	14 June 2000

ii) Taking Day as Single Fault Assumption i.e. Day will be having values varying from 1 to 31 and others will have nominal values.

Test Case	Month	Day	Year	Output
6	6	1	1960	31 May 1960
7	6	2	1960	1 June 1960
8	6	30	1960	29 June 1960
9	6	31	1960	Invalid day

iii) Taking Month as Single Fault Assumption i.e. Month will be having values varying from 1 to 12 and others will have nominal values.

Test Case	Month	Day	Year	Output
10	1	15	1960	14 Jan 1960
11	2	15	1960	14 Feb 1960
12	11	15	1960	14 Nov 1960
13	12	15	1960	14 Dec 1960

For the n variable to be checked Maximum of $4n + 1$ test case will be required.

Therefore, for $n = 3$, the maximum test cases are-

- **$4 \times 3 + 1 = 13$**

Boundary Value Analysis (BVA) :

Best for:

- ✓ Finding errors at edges
- ✓ Numeric ranges

What it does:

- Tests **minimum, maximum, just below & just above boundaries**

Strength:

- Very effective (most defects occur at boundaries)

Limitation:

- Does not cover internal values fully

2. Equivalence Partitioning (EP) :

Equivalence Partitioning is a **software testing technique** in which the input data is divided into **groups (partitions)** that are expected to behave the **same way**. One test case from each partition is sufficient to represent the entire group.

- If one value in a partition works correctly, **all values in that partition are assumed to work the same**
- Reduces the **number of test cases** while maintaining good coverage

Types of Equivalence Classes :

- **Valid equivalence classes**
- **Invalid equivalence classes**

Simple Example :

- **Requirement:** Accept marks from **0 to 100**

Equivalence partitions:

- Invalid: marks < 0
- Valid: 0 to 100
- Invalid: marks > 100

Test cases:

- -5 $\rightarrow$ invalid
- 50 $\rightarrow$ valid
- 120 $\rightarrow$ invalid

Real-Time Example of Equivalence Partitioning :

Example: Online Movie Ticket Booking – Age Validation

- Requirement:
- Children ticket: **Age < 12**
- Adult ticket: **Age 12 to 59**
- Senior citizen ticket: **Age ≥ 60**

Equivalence Partitions

Equivalence Class	Age Range	Type
EC1	Age < 0	✗ Invalid
EC2	0 – 11	✓ Valid
EC3	12 – 59	✓ Valid
EC4	≥ 60	✓ Valid

Representative Test Cases :

- -5 → Invalid input
- 8 → Child ticket
- 30 → Adult ticket
- 65 → Senior citizen ticket
- ✓ One value from each partition is enough
- ✓ All values in the same partition are treated the same

Why this is useful :

- Avoids testing **every possible age**
- Reduces test cases
- Makes testing efficient and systematic

Problem: Consider a Program for determining the Previous Date.

Input Conditions

- Day: 1–31
- Month: 1–12
- Year: 1900–2000

Equivalence Partitions

- **Day**
- EP1: 1–31 (valid)
- EP2: <1 (invalid)
- EP3: >31 (invalid)

- **Month**

- EP4: 1–12 (valid)
- EP5: <1 (invalid)
- EP6: >12 (invalid)

- **Year**

- EP7: 1900–2000 (valid)
- EP8: <1900 (invalid)
- EP9: >2000 (invalid)

Example : ATM Withdrawal

Requirement

- Minimum withdrawal: ₹100
- Maximum withdrawal: ₹10,000
- Only multiples of ₹100 allowed

Step 1: Equivalence Partitions

Partition Type	Amount
Valid	₹100–₹10,000 and multiple of 100
Invalid	< ₹100
Invalid	> ₹10,000
Invalid	Not a multiple of 100

Step 2: Test Cases

Test Case	Input Amount	Expected Output
TC1	₹500	Cash Dispensed
TC2	₹50	Error: Minimum limit ₹100
TC3	₹15,000	Error: Exceeds maximum limit
TC4	₹550	Error: Enter amount in multiples of 100

Equivalence Partitioning (EP) :

Best for:

- ✓ Reducing test cases
- ✓ Valid & invalid input classification

What it does:

- Divides input data into **valid and invalid partitions**
- Tests **one value from each partition**

Strength:

- Saves time
- Good for large input ranges

Limitation:

- May miss boundary defects

3. State Table–Based Testing :

What is State Table–Based Testing?

State table–based testing is a black-box testing technique used to verify system behavior where the output depends on the current state and the input.

- **Same input gives different output** in different states
- It is derived from a **state transition diagram** and represented in **tabular form**.

Steps to Perform State Table–Based Testing :

- Identify all **states**
- Identify all **inputs/events**
- Determine **valid and invalid transitions**
- Create the **state table**
- Derive **test cases** from the table
- Execute and verify outputs & state changes

Example: ATM Card System

System Description :

An ATM card system has the following behavior:

- Card can be **Inserted**
- PIN can be **Entered Correctly or Incorrectly**
- After **3 wrong PIN attempts**, card is **Blocked**

Step 1: Identify States

State ID	State Name
S0	Idle (No card)
S1	Card Inserted
S2	Authenticated
S3	Card Blocked

Step 2: Identify Inputs

Input ID	Input
I1	Insert Card
I2	Enter Correct PIN
I3	Enter Wrong PIN (≤ 2 attempts)
I4	Enter Wrong PIN (3rd attempt)
I5	Eject Card

Step 3: State Table

Current State	Input	Next State	Output
S0 (Idle)	Insert Card	S1	Card Accepted
S1 (Card Inserted)	Correct PIN	S2	Login Successful
S1	Wrong PIN (≤ 2)	S1	Error: Try Again
S1	Wrong PIN (3rd)	S3	Card Blocked
S2 (Authenticated)	Eject Card	S0	Card Returned
S3 (Blocked)	Any Input	S3	Card Retained

Step 4: Test Cases Derived from State Table

Test Case	Initial State	Input	Expected Output	Final State
TC1	S0	Insert Card	Card Accepted	S1
TC2	S1	Correct PIN	Login Successful	S2
TC3	S1	Wrong PIN (1st)	Error: Try Again	S1
TC4	S1	Wrong PIN (3rd)	Card Blocked	S3
TC5	S2	Eject Card	Card Returned	S0
TC6	S3	Any Input	Card Retained	S3

4. Decision Table–Based Testing :

Decision Table–Based Testing is a **black-box test design technique** used to test system behavior for **different combinations of input conditions**. It is especially useful when business rules are complex and multiple conditions affect the outcome.

Why Decision Table Testing is Used ?

- When **multiple conditions** exist
- When different **rules** apply for different combinations
- To ensure **complete coverage** of business logic
- To avoid missing important condition combinations

Types of Decision Tables :

Decision tables are mainly classified based on **how conditions and actions are represented.**

i). Limited-Entry Decision Table :

- Most commonly used
- Conditions have only **two values: Yes/No, True/False**
- Simple and easy to understand
- Widely used in **exam questions**

Ex : Login Validation

Problem Statement

A system allows a user to log in based on:

- Username is valid
- Password is valid

Decision Table :

Conditions	Rule 1	Rule 2	Rule 3	Rule 4

Username valid	T	T	F	F
Password valid	T	F	T	F

Actions				
Login allowed	X			
Show error msg		X	X	X

Test Cases Derived

TC	Username	Password	Expected Result
TC1	Valid	Valid	Login success
TC2	Valid	Invalid	Error shown
TC3	Invalid	Valid	Error shown
TC4	Invalid	Invalid	Error shown

ii). Extended-Entry Decision Table :

An **Extended-Entry Decision Table** is used when **conditions can have more than two values** (not just True/False).

- Conditions can have **multiple values**
- Used for **complex business rules**
- Reduces number of rules compared to limited-entry

Example: Student Grade Evaluation

Problem Statement

- Student grade is decided based on **marks obtained**.

Condition

- **Marks Range** (multiple values)

Actions

- Assign Grade A
- Assign Grade B
- Assign Grade C
- Fail

Extended-Entry Decision Table

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Marks (%)	≥85	70–84	50–69	<50
Actions				
Grade A	X			
Grade B		X		
Grade C			X	
Fail				X

Test Case Table			
TC ID	Input (Marks %)	Test Data (Example)	Expected Result
TC1	≥ 85	90	Grade A
TC2	70 – 84	75	Grade B
TC3	50 – 69	60	Grade C
TC4	< 50	40	Fail

iii). Mixed-Entry Decision Table :

A **Mixed-Entry Decision Table** contains:

- **Limited-entry conditions** → True / False
- **Extended-entry conditions** → Multiple values or ranges

Example: Online Shopping Discount System

- An **online shopping application** provides discounts to customers based on the following conditions:
- **Customer Type**
 - Premium
 - Regular
- **Purchase Amount**
 - Greater than or equal to ₹5000
 - Less than ₹5000

Discount Rules :

- Premium customers purchasing **₹5000 or more** receive a **20% discount**.
- Premium customers purchasing **less than ₹5000** receive a **10% discount**.
- Regular customers purchasing **₹5000 or more** receive a **10% discount**.
- Regular customers purchasing **less than ₹5000** receive **no discount**.
- **Customer Type** (Premium / Regular) → *Extended-entry*
- **Purchase Amount** ($\geq$ ₹5000 or $<$ ₹5000) → *Limited-entry*

Mixed-Entry Decision Table

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Customer Type	Premium	Premium	Regular	Regular
Purchase Amount ≥5000	T	F	T	F
Actions				
20% Discount	X			
10% Discount		X	X	
No Discount				X

Derived Test Cases

TC ID	Customer Type	Amount	Expected Result
TC1	Premium	₹6000	20% Discount
TC2	Premium	₹3000	10% Discount
TC3	Regular	₹7000	10% Discount
TC4	Regular	₹2000	No Discount

5. Cause-Effect graphing based Technique :

- **Cause–Effect Graphing** is a **black-box test design technique** used to identify **logical relationships** between **inputs (causes)** and **outputs (effects)** and then convert them into **decision tables and test cases**.
- It is especially useful when **multiple conditions combine to produce outputs**.
- **Cause** → Input condition (True/False)
- **Effect** → Output or system action
- **Graph** → Visual representation of logical relationships

Steps in Cause–Effect Graphing Technique :

Example Used: Login System

A user can log in **only if**:

- Username is valid
- Password is valid

Otherwise, an error message is displayed.

Step 1: Identify Causes and Effects

Causes (Inputs)

- **C1**: Username is valid
- **C2**: Password is valid

Effects (Outputs)

- **E1**: Login successful
- **E2**: Error message displayed

Step 2: Assign Logical Relationships

From the requirement:

- Login is successful **only when both inputs are valid** So:

$$E1 = C1 \text{ AND } C2$$

$$E2 = \text{NOT } (C1 \text{ AND } C2)$$

Step 3: Draw the Cause–Effect Graph (Logical View)

```
C1 (Username valid) ⊣  
                        |— AND —→ E1 (Login success)  
C2 (Password valid) ⊣  
  
E2 (Error message) = NOT (E1)
```

Step 4: Apply Constraints (If Any)

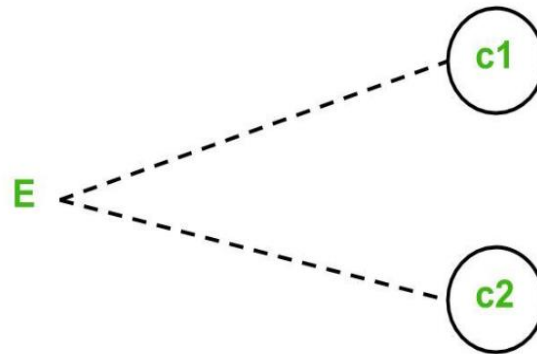
- No special constraints here
- All combinations of C1 and C2 are valid

Constraints :

- To represent some impossible combinations of causes or impossible combinations of effects, constraints are used. The following **constraints** are used in cause-effect graphs:

i) Exclusive constraint or E-constraint: This constraint exists between causes. It states that either c1 or c2 can be 1, i.e., c1 and c2 cannot be 1 simultaneously (If one condition is true, the other(s) **must be false**.).

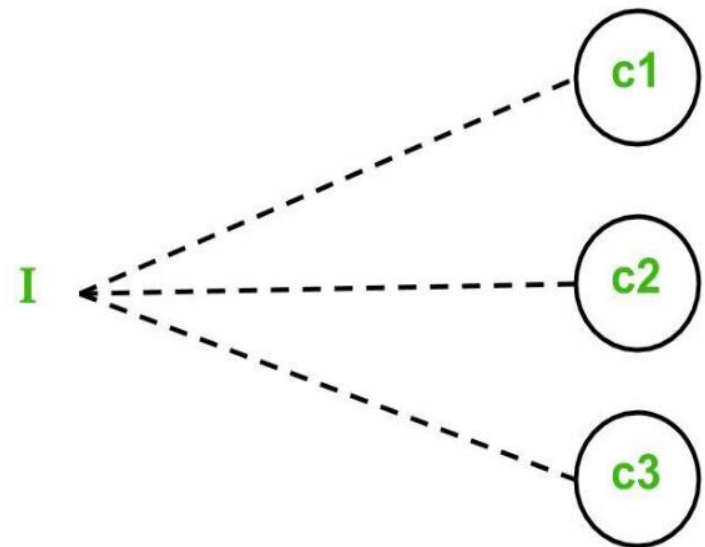
- Ex : A student **cannot pass and fail at the same time**.



ii) **Inclusive constraint or I-constraint:** This constraint exists between causes. It states that at least one of c_1 , c_2 and c_3 must always be 1, i.e., c_1 , c_2 and c_3 cannot be 0 simultaneously (**two or more conditions**

can be true at the same time, but at least one must be true..)

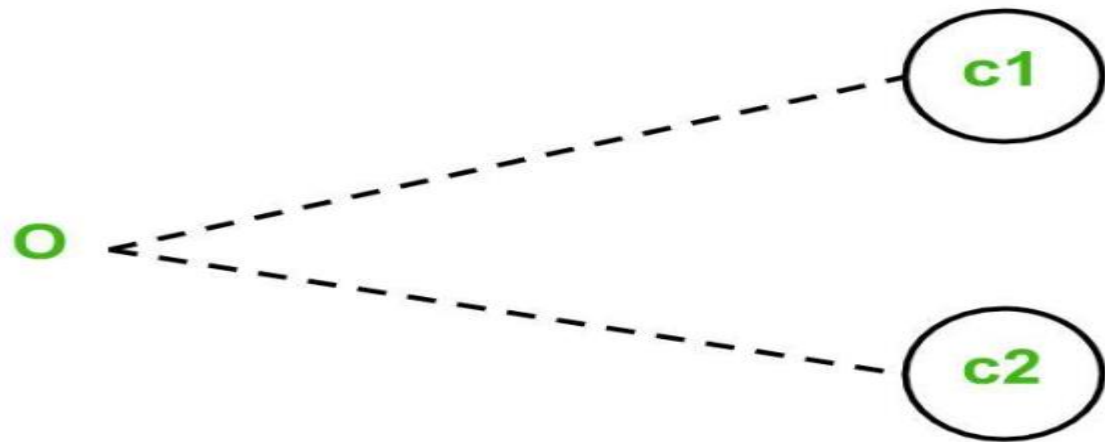
- Ex : A student is eligible for scholarship if:
- High academic score **OR**
- Sports quota
- **Both conditions can be true together**



iii) One and Only One constraint or

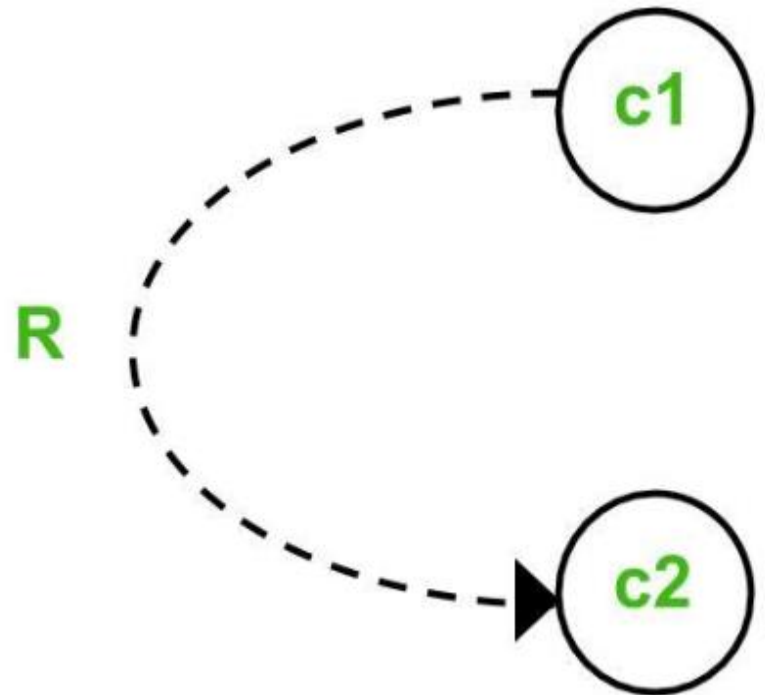
O-constraint: This constraint exists between causes. It states that one and only one of c1 and c2 must be 1. (**exactly one condition must be true, and all other conditions must be false.**)

Ex : The washing machine can run **only one mode at a time.**



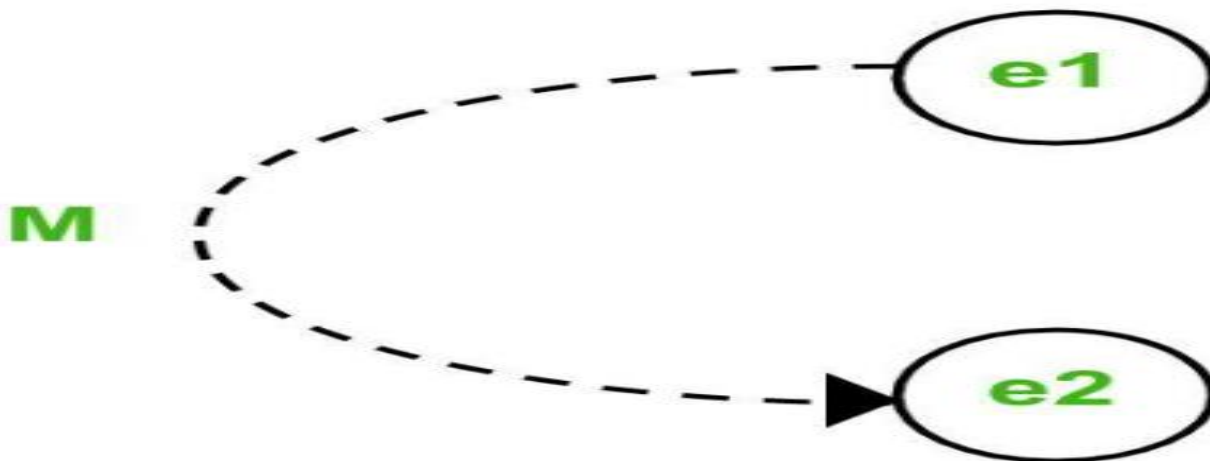
iv) Requires constraint or R-constraint: This constraint exists between causes. It states that for c_1 to be 1, c_2 must be 1. It is impossible for c_1 to be 1 and c_2 to be 0 (Condition A depends on Condition B).

Ex : Withdrawal can happen **only when card is inserted**.



v) Mask constraint or M-constraint: This constraint exists between effects. It states that if effect e_1 is 1, the effect e_2 is forced to be 0. (**Condition A is true, the effect of Condition B is ignored (masked), even if B is true.**)

Ex : If the order is **already shipped**, cancellation is **not allowed**, even if cancellation is requested.



Step 4: Apply Constraints (If Any)

- No special constraints here
- All combinations of C1 and C2 are valid

Step 5: Convert the Graph into a Decision Table

Conditions	Rule 1	Rule 2	Rule 3	Rule 4

Username valid	T	T	F	F
Password valid	T	F	T	F

Actions				
Login success	X			
Error message		X	X	X

Step 6: Derive Test Cases :

TC	Username	Password	Expected Result
TC1	Valid	Valid	Login successful
TC2	Valid	Invalid	Error message
TC3	Invalid	Valid	Error message
TC4	Invalid	Invalid	Error message

Step 7: Review and Finalize :

- All causes are covered
- All effects are tested
- No redundant test cases

6. Orthogonal array testing :

Orthogonal Array Testing is a **black-box test design technique** used to efficiently test systems with **multiple input parameters**, each having **multiple values**, by reducing the total number of test cases while still achieving **maximum coverage** of interactions.

Why Orthogonal Array Testing?

- When a system has many inputs, testing all combinations becomes impossible.

Example:

If there are **4 parameters**, each with **3 values**

→ Total combinations = $3^4 = \mathbf{81}$ **test cases**

- Using Orthogonal Array Testing, you may test only **9 or 16** cases while still covering **all pairwise combinations**.

Orthogonal Array (OA) :

- An **Orthogonal Array** is a mathematical matrix where:
- Rows → Test cases
- Columns → Input parameters
- Values → Possible values of parameters
- The array is constructed so that combinations are **evenly and systematically distributed**.

Example :

Problem :

A login system depends on:

Parameter	Values
Browser	Chrome, Firefox, Edge
OS	Windows, Linux, macOS
Network	WiFi, Mobile, LAN

Exhaustive Testing

- Each parameter has **3 values**.
- Total combinations= $3 \times 3 \times 3 = 27$ test cases
- Exhaustive testing requires **27 test cases**.

Pairwise Coverage :

- **Browser × OS**
- Chrome–Windows, Chrome–Linux, Chrome–macOS
- Firefox–Windows, Firefox–Linux, Firefox–macOS
- Edge–Windows, Edge–Linux, Edge–macOS
- **Browser × Network**
- Chrome–WiFi, Chrome–Mobile, Chrome–LAN
- Firefox–WiFi, Firefox–Mobile, Firefox–LAN
- Edge–WiFi, Edge–Mobile, Edge–LAN
- **OS × Network**
- Windows–WiFi, Windows–Mobile, Windows–LAN
- Linux–WiFi, Linux–Mobile, Linux–LAN
- macOS–WiFi, macOS–Mobile, macOS–LAN

Using Orthogonal Array (L9)

Test Case	Browser	OS	Network
TC1	Chrome	Windows	WiFi
TC2	Chrome	Linux	Mobile
TC3	Chrome	macOS	LAN
TC4	Firefox	Windows	Mobile
TC5	Firefox	Linux	LAN
TC6	Firefox	macOS	WiFi
TC7	Edge	Windows	LAN
TC8	Edge	Linux	WiFi
TC9	Edge	macOS	Mobile

- **pairwise combinations** are covered
- **Only 9 test cases instead of 27**(Exhaustive test)

Advantages

- Huge reduction in test cases
- Good defect detection rate
- Systematic and structured
- Cost-effective

Limitations

- Not suitable for **critical safety systems**
- May miss defects caused by **higher-order interactions**

7.Error Guessing testing :

Error Guessing is an **experience-based software testing technique** where the tester designs test cases based on **past experience, understanding, and knowledge of common defects**, rather than formal test design rules.

In **Error Guessing Testing**, helps a tester to:

- Predict where defects are likely to occur
- Identify risky areas of the application
- Design test cases based on previous failures

Example:

A tester checks **empty input fields** because such errors are common.

Why This Is Error Guessing?

- No rules like BVA or EP used
- Test cases created from **past experience**
- Focuses on **common user mistakes & system failures**

Possible Error-Guessing Test Cases

No.	Test Condition (Guessed Error)	Expected Result
1	Username field left blank	Error message displayed
2	Password field left blank	Error message displayed
3	Username contains only spaces	Login should fail
4	Password with very long length	Proper validation / error
5	Special characters in username (@#\$%)	Input rejected

Requirement-Based Testing (RBT) :

- **Requirement-Based Testing** is a **black-box testing method** in which **test cases are designed directly from the system requirements** to verify that every requirement is correctly implemented.
- **Requirement-Based Testing** is a testing method where test cases are derived from functional and non-functional requirements to ensure that the system meets all specified requirements.

Methods :

- Functional Requirements
- Non-Functional Requirements

Example :

Login System — Requirement-Based Testing

Functional Requirements for a Login System :

Step 1 :

Requirement Identification

RQ-01: The system shall allow users to log in using a valid username and password.

RQ-02: The system shall display an error message for invalid credentials.

RQ-03: The system shall not allow login when username or password is empty.

RQ-04: After 3 consecutive failed attempts, the account shall be temporarily locked.

RQ-05: On successful login, the system shall redirect to the dashboard.

Step 2 :

Test Scenarios (Derived from Requirements)

- TS-01: Verify login with valid credentials
- TS-02: Verify login with invalid credentials
- TS-03: Verify validation for empty fields
- TS-04: Verify account lock after multiple failures
- TS-05: Verify successful redirection after login

Step 3 : Test Cases

TC ID	Requirement ID	Test Condition	Input	Expected Result
TC-01	RQ-01	Valid login	Valid username & password	Login successful
TC-02	RQ-02	Invalid password	Valid username + wrong password	Error message
TC-03	RQ-02	Invalid username	Wrong username + valid password	Error message
TC-04	RQ-03	Empty username	Username empty	Validation message
TC-05	RQ-03	Empty password	Password empty	Validation message
TC-06	RQ-04	3 failed attempts	Wrong password 3 times	Account locked
TC-07	RQ-05	Post login	Valid credentials	Redirect to dashboard

Step 4 : Requirement Traceability Matrix (RTM)

Requirement ID	Test Case IDs
RQ-01	TC-01
RQ-02	TC-02, TC-03
RQ-03	TC-04, TC-05
RQ-04	TC-06
RQ-05	TC-07

Non-Functional Requirements for a Login System :

1. Performance Requirements :

- Login response time shall be ≤ 2 seconds under normal load.
- System shall support at least 10,000 concurrent login requests.
- Authentication processing time shall not exceed 500 ms.

2. Security Requirements :

- Passwords shall be stored using **strong encryption / hashing**.
- Login attempts shall be **limited to 3–5 failures** before logout.
- CAPTCHA shall be enabled after multiple failed attempts.
- System shall prevent **SQL injection and XSS attacks**.
- Session ID shall be **random and secure**.

3. Usability Requirements :

- Error messages shall be **clear and user-friendly**.
- Password rules shall be **displayed to users**.
- Login page shall be accessible within **2 clicks**.
- Support **show/hide password** option.

4. Reliability Requirements :

- Login service availability shall be **99.9% uptime**.
- System shall recover automatically after failure.
- No loss of authentication data during crashes.

5. Scalability Requirements :

- System shall handle **increased users without performance degradation.**
- Login response time shall remain stable during peak hours.

6. Compatibility Requirements :

- Login system shall work on **all major browsers.**
- Support **desktop, tablet, and mobile devices.**
- Compatible with **multiple operating systems.**

7. Accessibility Requirements :

- Login page shall support **keyboard navigation.**
- Screen readers shall be able to read login fields.
- Follow **WCAG accessibility guidelines.**

8. Maintainability Requirements :

- Login configuration changes shall not require code changes.
- Logs shall be maintained for **login success/failure**.
- Easy to update password policies.

Functional Testing Tools

- **Functional Testing Tools** are software tools used to check whether an application works according to its functional requirements.
- They verify **what the system does**, not how it is built.
- Functional testing tools help testers confirm that **features like login, search, payment, and registration work correctly**.

1. Selenium

- Selenium is an **open-source automation tool** mainly used for **functional testing of web applications**.
- It automates browsers by simulating real user actions like clicking buttons, entering text, and navigating web pages.
- Supports multiple browsers (Chrome, Firefox, Edge)
- Supports multiple programming languages
- Integrates with CI/CD tools **CI (Continuous Integration)**: Automatically integrates and tests code changes.
- **CD (Continuous Delivery/Deployment)**: Automatically delivers or deploys code to production.
- **Advantages**
- ✓ Free and open source
- ✓ Large community support
- ✓ Cross-browser testing

Example

- Testing login functionality by entering credentials and verifying the dashboard page.

2. UFT (Unified Functional Testing – formerly QTP)

- UFT is a **commercial functional testing tool** widely used in **enterprise applications**.
- Uses **object-based testing**, identifying UI elements and validating their properties using VBScript.
- **Key Features**
 - Keyword-driven testing
 - Data-driven testing
 - Strong reporting
- **Advantages**
 - ✓ Suitable for large systems
 - ✓ Good technical support
- **Example**
 - Testing banking transactions or ERP workflows.

3. TestComplete

- TestComplete is a **commercial automation tool** used for **web, desktop, and mobile functional testing**.
- Supports record-and-playback and scripting approaches.
- **Key Features**
- Keyword-driven testing
- Multiple scripting languages
- Strong object recognition
- **Advantages**
- ✓ Easy test creation
- ✓ Supports desktop applications
- **Example**
- Testing payroll or inventory systems.

4. Appium

Appium is an **open-source functional testing tool** for **mobile applications**.

- Uses mobile OS automation frameworks to test native, hybrid, and mobile web apps.
- **Key Features**
- Cross-platform testing (Android & iOS)
- No app modification needed
- **Advantages**
- ✓ Reusable test scripts
- ✓ Open source
- **Example**
- Testing mobile login, notifications, and navigation.

5. Cypress

- Cypress is an **open-source JavaScript-based tool** for testing **modern web applications**.
- Runs directly in the browser and provides real-time execution.
- **Key Features**
- Automatic waits
- Easy debugging
- **Advantages**
- ✓ Fast execution
- ✓ Simple setup
- **Limitation**
- ✗ Limited browser support compared to Selenium
- **Example**
- Testing form validation and error messages.

6. Playwright

- Playwright is an **open-source end-to-end testing tool** for web applications.
- Automates browsers using modern APIs and supports parallel testing.
- **Key Features**
- Cross-browser support
- Reliable automation
- **Advantages**
- ✓ High performance
- ✓ Supports multiple languages
- **Example**
- Testing responsive web layouts.

7. Katalon Studio

- Katalon Studio is a **low-code functional testing tool** built on Selenium and Appium.
- Provides ready-made keywords and built-in reports.
- **Key Features**
 - Web, mobile, API testing
 - Easy learning curve
- **Advantages**
 - ✓ Beginner-friendly
 - ✓ All-in-one solution
- **Example**
 - Testing e-commerce checkout flow.

8. SoapUI

- SoapUI is a **functional testing tool for APIs**.
- Sends REST/SOAP requests and validates responses.
- **Key Features**
- Data-driven API testing
- Security and performance checks
- **Advantages**
- ✓ Strong API validation
- ✓ Easy interface
- **Example**
- Verifying API response data and status codes.

9. Ranorex

- **Ranorex is a commercial functional test automation tool used for testing desktop, web, and mobile applications.**
- Ranorex uses **powerful object recognition** to identify UI elements (buttons, text boxes, menus) and automate user actions on them.
- It supports:
- **Codeless testing** (record & playback)
- **Script-based testing** (C# or VB.NET)

10. Tricentis Tosca

- **Tricentis Tosca** is a **commercial functional test automation tool** that uses a **model-based testing approach** instead of traditional scripting.
- It is widely used for **enterprise applications** such as **Web, SAP, API, Desktop, and Mobile** systems.
- Tosca creates **business models** of application screens and processes.
- Test cases are designed by **configuring modules**, not by writing code.
- This reduces test maintenance when the application changes.

Non-Functional Requirement (NFR) Tools

- **Non-functional requirement tools** are software tools used to test **how well** a system performs rather than **what it does**.
- They validate quality attributes such as They focus on **quality attributes** like:

Performance – How fast the system responds ?

Security – How protected the system is ?

Usability – How easy it is to use ?

Reliability – How stable it is ?

Scalability – Can it handle growth?

Compatibility – Works across browsers, OS, devices

1. Performance Testing Tools

- **Performance testing tools** are software tools used to test how a system **performs under load**.

They measure:

- **Response time** – How fast the system responds
- **Throughput** – How many transactions it can handle
- **Stability** – How reliable the system is under heavy load
- It ensure the application is **fast, stable, and scalable** under different workloads.

1. JMeter

- **Type:** Open-source
- **Purpose:** Load and stress testing
- **How it works:** Simulates multiple virtual users accessing a system and measures response time, throughput, and latency.
- **Example:** Testing an e-commerce website to see how it behaves when **1,000 users try to checkout simultaneously**.

2. LoadRunner

- **Type:** Commercial
- **Purpose:** Enterprise performance testing
- **How it works:** Creates thousands of virtual users to identify performance bottlenecks.
- **Example:** Testing an airline booking system to handle **peak season traffic** without crashing.

3. Gatling

- **Type:** Open-source
- **Purpose:** High-performance load testing for APIs
- **How it works:** Uses scripts to simulate thousands of users and measure system performance.
- **Example:** Testing a payment gateway API under **high transaction load**.

4.BlazeMeter

- **Type:** Cloud-based
- **Purpose:** Scalable performance and load testing
- **How it works:** Allows performance testing from **multiple geographic locations.**
- **Example:** Checking response time of a global news website when accessed by users from different countries.

5.NeoLoad

- **Type:** Commercial
- **Purpose:** Load, stress, and scalability testing
- **How it works:** Creates realistic user scenarios to measure system behavior under load.
- **Example:** Testing mobile banking app to ensure it handles **10,000 concurrent logins** smoothly.

2. Security Testing Tools

- **Security testing tools** are used to identify **vulnerabilities, threats, and risks** in a software application to protect data and ensure secure operation.

They check issues like:

- Unauthorized access
- SQL Injection
- Cross-Site Scripting (XSS)
- Session hijacking

1. OWASP ZAP (Zed Attack Proxy)

- **Type:** Open-source
- **Purpose:** Web application security testing
- OWASP ZAP scans web applications to find common security vulnerabilities such as SQL injection and XSS.
- **Example:**
Testing a login page to ensure it is not vulnerable to SQL injection attacks.

2. Burp Suite

- **Type:** Free + Commercial
- **Purpose:** Penetration testing
- Burp Suite intercepts and analyzes HTTP/HTTPS traffic between the browser and the server to identify security flaws.
- **Example:**
Testing whether user sessions can be hijacked by manipulating cookies.

3. Acunetix

- **Type:** Commercial
- **Purpose:** Automated vulnerability scanning
- Acunetix scans websites to detect security vulnerabilities like broken authentication and insecure configurations.
- **Example:**
Scanning an e-commerce website to detect payment-page vulnerabilities.

4. Nessus

- **Type:** Commercial
- **Purpose:** Vulnerability assessment
- Nessus identifies security weaknesses in servers, networks, and applications.
- **Example:**
Checking whether a server has outdated software with known security risks.

5. SQLMap

- **Type:** Open-source
- **Purpose:** SQL Injection testing
- SQLMap automates detection and exploitation of SQL injection vulnerabilities.
- **Example:**
Testing whether database queries can be manipulated using malicious SQL inputs.

3. Usability & Accessibility Testing Tools

- **Usability and accessibility testing tools** are used to check how **easy and inclusive** a software application is for users, including people with disabilities.
- **Usability** → Ease of use, user experience, navigation
- **Accessibility** → Support for users with visual, hearing, or motor impairments

1. Lighthouse

- **Type:** Free (Google)
- **Purpose:** Usability, accessibility, and performance testing
- Lighthouse analyzes web pages and provides scores and suggestions for improving **accessibility, usability, and performance.**
- **Example:**
Checking whether a website loads fast and uses proper contrast for text readability.

2. Axe

- **Type:** Open-source
- **Purpose:** Accessibility testing
- Axe checks web applications against **WCAG accessibility standards**, identifying missing labels, color contrast issues, and screen-reader problems.
- **Example:**
Verifying whether form fields have labels for screen readers.

3. WAVE (Web Accessibility Evaluation Tool)

- **Type:** Free
- **Purpose:** Accessibility evaluation
- WAVE visually highlights accessibility issues directly on the web page.
- **Example:**
Identifying missing alternative text for images.

4. Hotjar

- **Type:** Commercial
- **Purpose:** Usability testing
- Hotjar records user interactions like clicks, scrolls, and mouse movements to analyze user behavior.
- **Example:**
Understanding where users struggle during checkout.

5. Screen Reader Tools (NVDA / JAWS)

- **Type:** Free / Commercial
- **Purpose:** Accessibility testing
- These tools simulate how visually impaired users experience an application.
- **Example:**
Testing whether a website can be navigated using only a screen reader.

4. Compatibility / Cross-Browser Tools

- **Compatibility (Cross-Browser) testing tools** are used to verify that an application works **consistently across different browsers, operating systems, devices, and screen sizes**.
- They ensure the application looks and behaves the same for all users.

1. BrowserStack

- **Type:** Commercial (cloud-based)
- **Purpose:** Cross-browser and cross-device testing
- BrowserStack allows testers to test websites and mobile apps on **real browsers and real devices** without installing them locally.
- **Example:**
Checking whether a website displays correctly on **Chrome, Firefox, Safari, and Edge**.

2. Sauce Labs

- **Type:** Commercial
- **Purpose:** Automated and manual cross-browser testing
- Sauce Labs supports automated testing across multiple browsers, operating systems, and mobile devices in the cloud.
- **Example:**
Running automated Selenium tests on different browser versions.

• 3. CrossBrowserTesting

- **Type:** Commercial
- **Purpose:** Cross-browser compatibility testing
- Provides live testing and screenshots across many browsers and devices.
- **Example:**
Taking screenshots of a webpage on different browser versions to compare layout.

4. LambdaTest

- **Type:** Commercial
- **Purpose:** Browser and OS compatibility testing
- LambdaTest allows testing across multiple browser-OS combinations using manual and automated tests.
- **Example:**
Testing a web application on **Windows + Chrome** and **macOS + Safari**.

5. Reliability / Stability Testing Tools

- **Reliability and stability testing tools** are used to check whether a software application **runs continuously without failure** over a long period of time.
- They ensure the system is **stable, dependable, and error-free** under normal and heavy usage.

1. Apache JMeter

- **Type:** Open-source
- **Purpose:** Load, stress, and endurance (soak) testing
- JMeter runs an application under continuous load for a long duration to identify memory leaks, crashes, or performance degradation.
- **Example:**
Testing a banking application for **24 hours** to ensure it does not crash during continuous transactions.

2. LoadRunner

- **Type:** Commercial
- **Purpose:** Enterprise reliability and stability testing
- LoadRunner simulates thousands of virtual users over extended periods to check system stability and reliability.
- **Example:**
Testing an airline reservation system for **continuous booking operations**.

3. NeoLoad

- **Type:** Commercial
- **Purpose:** Long-duration performance and stability testing
- NeoLoad helps detect failures and resource leaks during prolonged system usage.
- **Example:**
Running a **72-hour endurance test** on an e-commerce website.

4. Gatling

- **Type:** Open-source
- **Purpose:** Reliability testing for APIs
- Gatling ensures APIs remain stable under constant load.
- **Example:**
Testing a payment gateway API to ensure **no downtime** during continuous usage.

6. Scalability & Stress Testing Tools

- **Scalability and stress testing tools** are used to evaluate how a software application behaves when the **workload is increased beyond normal limits**.
- **Scalability testing** → Checks whether the system can **handle growing users or data**
- **Stress testing** → Checks system behavior under **extreme load or breaking point**

1. Apache JMeter

- **Type:** Open-source
- **Purpose:** Scalability and stress testing
- JMeter simulates a large number of users and gradually increases load to check system scalability and failure points.
- **Example:**
Testing an e-commerce website by increasing users from **1,000 to 10,000** during a flash sale.

2. LoadRunner

- **Type:** Commercial
- **Purpose:** Enterprise-level scalability and stress testing
- LoadRunner creates thousands of virtual users to identify system limits and bottlenecks.
- **Example:**
Stress testing an airline booking system during peak holiday traffic.

3. Gatling

- **Type:** Open-source
- **Purpose:** High-performance stress testing
- Gatling uses lightweight virtual users to test how systems behave under extreme loads.
- **Example:**
Testing a payment API by sending **high transaction volumes per second**.

4. NeoLoad

- **Type:** Commercial
- **Purpose:** Scalability testing for web and mobile apps
- NeoLoad measures how well an application scales while load increases.
- **Example:**
Testing a mobile banking app to support **10× user growth**.